

Module 6

Complex Testbenches

Learning objectives

- Build complex multi-agent testbenches with protocol verification

Prerequisites

- See module README

Learning path

Learning Path

Diagram: Learning Path

(render with mmdc when available)

flowchart LR

T1["Multi-Agent Environments"]

T2["Protocol Verification"]

T1 --> T2

T3["Testbench Architecture Patte"]

T2 --> T3

Build complex multi-agent testbenches with protocol verification

Overview

- This module focuses on building complex verification environments with multiple agents, protocol ve...

How to learn this module

Suggested learning path

- Module 5 multi-agent and virtual sequence patterns carry forward — AXI adds real protocol timing
- Read ``module6/dut/protocols/axi4_lite_slave.v``
AW/W/B/AR/R channel interfaces before stimulus
- Run ``--multi-agent`` and ``--protocol`` examples before
``--protocol-checkers`` and scoreboard labs
- Understand initiator vs slave roles: DUT is slave;
TB agents play masters/memory/peripheral
- Architecture example documents env packaging — study
before ``test_complex_testbench.py``

Design architecture

1. Protocol DUT architecture

- ``module6/dut/protocols/axi4_lite_slave.v`` — five-channel AXI4-Lite slave + memory interface
- Separate address/write/read response channels exercise real protocol timing
- Multi-agent TB can attach initiator agents while DUT acts as slave

Rtl Architecture

Diagram: Rtl Architecture
(render with mmdc when available)
flowchart TB
subgraph axi["axi4_lite_slave"]
AW[AW channel]
W[W channel]
B[B channel]
AR[AR channel]

2. Complex testbench architecture

- Multiple agents (e.g., master, memory, low-speed peripheral) under one `uvm\_env`
- Protocol checkers validate handshake rules independent of scoreboard
- Layered scoreboards: per-agent checks + system-level consistency
- Architecture example documents reusable env patterns and package boundaries

Verification Architecture

Diagram: Verification Architecture
(render with mmdc when available)
flowchart TB
ENV[uvm_env] --> MA[Master agent]
ENV --> SA[Slave / memory agent]
ENV --> PC[Protocol checker]
MA --> DUT[AXI slave]
SA --> DUT

3. Execution pipeline

- Build: Verilator compiles AXI slave RTL; env instantiates master/memory/peripheral agents and check...
- Connect: each agent monitor feeds protocol checker and scoreboard analysis ports
- Sim: virtual sequences configure agents then launch concurrent read/write traffic to slave
- Check: protocol checker asserts channel rules; scoreboard correlates addr/data/response across ports

4. Data flow

- Virtual sequences coordinate cross-agent scenarios (config then traffic)
- Monitors on each interface feed checkers and coverage
- Reference models optional for memory content golden checks
- System scoreboard ties master observations to slave memory updates

DUT — AXI4-Lite slave

```
/**
 * Module 6: AXI4-Lite Slave
 *
 * A simplified AXI4-Lite slave for protocol verification.
 *
 * Ports:
 *   ACLK:      Clock signal
 *   ARESETn:   Active-low reset
 *
 * Write Address Channel:
 *   AWVALID:   Write address valid
 *   AWREADY:   Write address ready
 *   AWADDR:    Write address
 *   AWPROT:    Write protection type
 *
 * Write Data Channel:
 *
 * # ...
```

Multi-agent testbench

```
"""
Module 6 Test: Complex Testbench Test
Complete testbench demonstrating complex multi-agent environment.
"""

import cocotb
from cocotb.clock import Clock
from cocotb.triggers import Timer, RisingEdge
from pyuvvm import *
# Explicitly import uvm_seq_item_pull_port - it may not be exported by
from pyuvvm import *
# Try multiple possible import paths
_uvm_seq_item_pull_port = None
try:
    # First try: check if it's in the namespace after from pyuvvm import
    *
    _uvm_seq_item_pull_port = globals()['uvm_seq_item_pull_port']
```

Key files to study

Open these in the repo

- ``module6/dut/protocols/axi4_lite_slave.v`` — five-channel AXI4-Lite slave with memory port
- ``module6/examples/protocol/protocol_example.py`` — directed AXI4-Lite transactions
- ``module6/examples/multi_agent/multi_agent_example.py`` — multiple agents under one env
- ``module6/examples/protocol_checkers/protocol_checker_example.py`` — handshake rule assertions
- ``module6/tests/pyuvm_tests/test_complex_testbench.py`` — system-level multi-agent regression
- ``scripts/module6.sh`` — routes protocol, checker, scoreboard, and architecture example flags

Verification & testing methods

1. Protocol verification methods

- Protocol checker asserts
AW/W/B/AR/R channel rules and
ordering
- Directed AXI transactions in
`protocol\_example.py` and
`test\_complex\_testbench`
- Multi-agent tests verify
concurrent masters do not
violate slave assumptions

Testing Methods

Diagram: Testing Methods

(render with mmdc when available)

flowchart TD

CHK[protocol checker rules] --> TR[AXI transactions]

TR --> DUT[DUT]

DUT --> SB[System scoreboard]

SB --> P{Consistent?}

2. System-level checking

- Scoreboard correlates transactions across ports (address, data, response)
- Architecture tests validate env wiring before long regressions
- Debug: transaction logs, UVM verbosity, Verilator waveforms

3. Step-by-step lab execution

- 1. Multi-agent wiring: ``./scripts/module6.sh --multi-agent``
- 2. Directed protocol: ``./scripts/module6.sh --protocol``
- 3. Protocol checkers: ``./scripts/module6.sh --protocol-checkers``
- 4. Scoreboards/architecture: ``./scripts/module6.sh --scoreboards --architecture``
- 5. System regression: ``./scripts/module6.sh --pyuvm-tests`` — full complex testbench pass

4. Closure

- ``./scripts/module6.sh --pyuvm-tests``; exercises extend multi-agent and protocol checkers
- Assessment: multi-agent envs, protocol verification, complex scoreboards, architecture patterns
- Explain how checker failures differ from scoreboard data mismatches

Syllabus topics

1. Multi-Agent Environments (1/3)

- Environment Architecture
- Multiple agent coordination
- Agent communication
- Environment hierarchy
- Environment patterns
- Agent Coordination

1. Multi-Agent Environments (2/3)

- Master-slave agents
- Peer-to-peer agents
- Multi-channel agents
- Agent synchronization
- Environment Patterns
- Layered environments

1. Multi-Agent Environments (3/3)

- Hierarchical environments
- Flat environments
- Mixed environments

2. Protocol Verification (1/4)

- Protocol Verification Overview
- What is protocol verification?
- Protocol compliance
- Protocol checking
- Protocol coverage
- AXI Protocol Verification

2. Protocol Verification (2/4)

- AXI protocol basics
- AXI4-Lite agent
- AXI4 agent
- AXI protocol checker
- Custom Protocol Verification
- Protocol definition

2. Protocol Verification (3/4)

- Protocol agent creation
- Protocol checker implementation
- Protocol coverage
- Protocol Checkers
- Checker implementation
- Protocol rule checking

2. Protocol Verification (4/4)

- Error detection
- Protocol compliance

3. Testbench Architecture Patterns (1/3)

- Layered Testbench
- Abstraction layers
- Layer communication
- Layer organization
- Layer patterns
- Reusable Components

3. Testbench Architecture Patterns (2/3)

- Component design
- Component reuse
- Component libraries
- Component patterns
- Testbench Templates
- Standard templates

3. Testbench Architecture Patterns (3/3)

- Template customization
- Template patterns
- Template best practices

4. Debugging and Analysis (1/4)

- UVM Debugging Techniques
- Phase debugging
- Component debugging
- Transaction debugging
- Configuration debugging
- Transaction Recording

4. Debugging and Analysis (2/4)

- Transaction logging
- Transaction tracing
- Transaction replay
- Transaction analysis
- Waveform Analysis
- VCD/FST generation

4. Debugging and Analysis (3/4)

- Waveform viewing
- Signal tracing
- Timing analysis
- Log Analysis
- Log parsing
- Error analysis

4. Debugging and Analysis (4/4)

- Performance analysis
- Coverage analysis

5. Multi-Channel Verification (1/2)

- Channel Coordination
- Multiple channels
- Channel synchronization
- Channel independence
- Channel patterns
- Bidirectional Interfaces

5. Multi-Channel Verification (2/2)

- Master-slave interfaces
- Bidirectional agents
- Interface coordination
- Interface patterns

6. Performance Verification (1/2)

- Performance Monitoring
- Performance metrics
- Performance collection
- Performance analysis
- Performance reporting
- Throughput Analysis

6. Performance Verification (2/2)

- Throughput measurement
- Bandwidth analysis
- Latency measurement
- Performance optimization

7. Error Injection and Recovery (1/2)

- Error Injection
- Error scenarios
- Error injection mechanisms
- Error patterns
- Error testing
- Recovery Testing

7. Error Injection and Recovery (2/2)

- Recovery scenarios
- Recovery verification
- Recovery patterns
- Recovery testing

8. Testbench Integration (1/2)

- Component Integration
- Integration strategies
- Integration testing
- Integration patterns
- Integration best practices
- System Integration

8. Testbench Integration (2/2)

- System-level integration
- Integration verification
- Integration patterns
- Integration challenges

9. Advanced Scoreboarding (1/2)

- Multi-Channel Scoreboards
- Multiple channel checking
- Channel coordination
- Scoreboard patterns
- Scoreboard optimization
- Time-Based Matching

9. Advanced Scoreboarding (2/2)

- Temporal matching
- Time windows
- Matching algorithms
- Matching patterns

10. Testbench Maintenance (1/2)

- Code Organization
- File organization
- Class organization
- Namespace management
- Documentation
- Version Control

10. Testbench Maintenance (2/2)

- Git workflows
- Branching strategies
- Code review
- Release management

Command reference highlights

Protocol and agent examples

- ``./scripts/module6.sh --multi-agent --protocol`` — agent fabric and directed AXI traffic
- ``./scripts/module6.sh --protocol-checkers`` — independent protocol rule validation
- ``./scripts/module6.sh --scoreboards --architecture`` — layered checking and env packaging

Complex testbench regression

- ``./scripts/module6.sh --pyuvvm-tests`` —
 ``test_complex_testbench`` full env
- ``cd module6/tests/pyuvvm_tests && make SIM=verilator``
 `TEST=test_complex_testbench``
- ``./scripts/module6.sh --skip-examples --pyuvvm-tests``
 — system test when components are validated

Debug concurrent traffic

- Transaction logs plus UVM verbosity — trace AW/W ordering vs W data beats
- Protocol checker failures often precede scoreboard mismatches — fix checker errors first
- Waveforms on ``awvalid/awready``, ``wvalid/wready`` for handshake stalls

Hands-on examples

Module 6 orchestrator

```
$ ./scripts/module6.sh --help
```

Terminal: module6_check

```
[[0;36m===== [[0m
[[0;36mModule 6: Complex Testbenches [[0m
[[0;36m===== [[0m
```

Usage: ./scripts/module6.sh [OPTIONS]

Module 6: Complex Testbenches

This script runs examples and tests for Module 6.

OPTIONS:

Examples:

--multi-agent	Run multi-agent environment examples
--protocol	Run protocol verification examples
--protocol-checkers	Run protocol checker examples
--scoreboards	Run multi-channel scoreboard examples
--architecture	Run testbench architecture examples
--all-examples	Run all examples (default)
--skip-examples	Skip all examples

Tests:

--pyuvvm-tests	Run pyuvvm tests
----------------	------------------

Environment:

--venv DIR	Virtual environment directory (default: .venv)
--no-venv	Don't use virtual environment
--sim SIMULATOR	Simulator to use (default: verilator)

Other:

--help, -h	Show this help message
------------	------------------------

EXAMPLES:

```
# Run all examples
./scripts/module6.sh
```

```
# Run specific examples
```

Exercise scaffold

```
./scripts/module6.sh --scaffold
```

Demo: Multi-Agent Environment

```
$ .venv/bin/python module6/examples/multi_agent/multi_agent_example.py
```

Terminal: ex01_multi_agent

Traceback (most recent call last):

File "/mnt/d/proj/universal-verification-methodology/learn_uvm_pyuvvm/module6/examples/multi_ag
from pyuvvm import *

ModuleNotFoundError: No module named 'pyuvvm'

Demo: Protocol Verification

```
$ .venv/bin/python module6/examples/protocol/protocol_example.py
```

Terminal: ex02_protocol

Traceback (most recent call last):

File "/mnt/d/proj/universal-verification-methodology/learn_uvm_pyuvvm/module6/examples/protocol

from pyuvvm import *

ModuleNotFoundError: No module named 'pyuvvm'

Demo: Protocol Checker

```
$ .venv/bin/python  
    module6/examples/protocol_checkers/protocol_checker_example.py
```

Terminal: ex03_protocol_checkers

Traceback (most recent call last):

File "/mnt/d/proj/universal-verification-methodology/learn_uvm_pyuvvm/module6/examples/protocol
from pyuvvm import *

ModuleNotFoundError: No module named 'pyuvvm'

Demo: Multi-Channel Scoreboard

```
$ .venv/bin/python  
    module6/examples/scoreboards/multi_channel_scoreboard_example.py
```

Terminal: ex04_scoreboards

Traceback (most recent call last):

File "/mnt/d/proj/universal-verification-methodology/learn_uvm_pyuvmm/module6/examples/scoreboa
from pyuvmm import *

ModuleNotFoundError: No module named 'pyuvmm'

Demo: Testbench Architecture

```
$ .venv/bin/python  
    module6/examples/architecture/architecture_example.py
```

Terminal: ex05_architecture

Traceback (most recent call last):

File "/mnt/d/proj/universal-verification-methodology/learn_uvm_pyuvvm/module6/examples/architec
from pyuvvm import *

ModuleNotFoundError: No module named 'pyuvvm'

Practice & assessment

What you should know (1/3)

- Design multi-agent environments
- Implement protocol verification
- Apply testbench architecture patterns
- Debug complex testbenches
- Analyze simulation results
- Coordinate multiple channels

What you should know (2/3)

- Monitor performance
- Integrate components
- Maintain testbenches
- Apply industry best practices
- Multiple agents
- Agent coordination

What you should know (3/3)

- Environment structure

Exercises

- Multi-Agent Environment
- Protocol Verification
- Testbench Architecture
- Debugging
- Performance Analysis

Assessment checklist

- Can design multi-agent environments
- Can implement protocol verification
- Understands architecture patterns
- Can debug complex testbenches
- Can analyze simulation results
- Can coordinate multiple channels

Summary & next steps

- Build complex multi-agent testbenches with protocol verification
- Complete module6/CHECKLIST.md
- Review module6/EXAMPLES.md and run each lab
- Next: Next module in course